Estatly Guide: Clients

The Estatly clients feature explained file by file and line by line: how clients are fetched and shaped, how mutations keep every screen fresh, how the board, the card and the form are built, and why. With the decisions behind them, the changes made in review, interview questions and answers, and vocabulary.

Branch: refactor/clients. Updated November 6, 2026. Matches the code in your repo after this review.

Contents

- 1. The big picture
- 2. pages/Clients.tsx
- 3. api/apiClients.ts
- 4. The five data hooks
- 5. useClientsBoard.ts
- 6. ClientsBoard.tsx
- 7. ClientsSkeleton.tsx
- 8. ClientCard.tsx
- 9. ClientActions.tsx
- 10. useClientForm.ts
- 11. ClientForm.tsx
- 12. Decisions and changes
- 13. Still open
- 14. Interview questions
- 15. Vocabulary
- 16. Self-test

1. The big picture

The clients page is the agency's **sales pipeline**. Every client sits in one stage: Active, Closed won, Closed lost or Inactive. Agents add clients, edit their details, move them between stages and, rarely, delete one added by mistake.

How the data flows

Layer	File	Job
Page	pages/Clients.tsx	Places the feature component
Component	ClientsBoard, ClientCard, ClientActions, ClientForm	Markup only
Board hook	useClientsBoard	Data, mutations, sheet state, grouping
Data hooks	useClients, useCreateClient, useEditClient, useMoveClient, useDeleteClient	Cache and refresh
API	apiClients.ts	Validate, query, shape, catch silent RLS blocks
Database	Supabase + RLS + ON DELETE RESTRICT	The rules that cannot be bypassed

What each role sees

Role	Clients visible	Agent line on cards
Admin, manager	Every client in the agency	Shown: who owns the lead
Agent	Only their own clients (RLS)	Hidden: it would always be them

Edit vs move: two different actions

- **Edit** changes the details (name, phone, budget). It opens a form in a sheet.
- **Move** changes the pipeline stage. It is one tap in the card's menu.
- They have different hooks, different API functions and different schemas, so the names must make the difference obvious: useEditClient and useMoveClient.

2. pages/Clients.tsx

```
import ClientsBoard from "@/features/clients/ClientsBoard"

export default function Clients() {
  return <ClientsBoard />
}
```

- **Pages stay thin.** Routes import from pages/, and the logic lives in features/clients.
- This is where page-level things would go later, such as a document title or a page layout wrapper.
- Same pattern as the dashboard page, so every route reads the same way.

3. api/apiClients.ts

Every function in this file talks to Supabase. Components and hooks never call supabase directly.

getClientsApi

```
const { data, error } = await supabase
  .from("clients")
  .select(
    "*, agent:user_profiles!clients_assigned_agent_id" +
    "_user_profiles_fkey(full_name, avatar, email), " +
    "viewings(count)"
  )
  .order("created_at", { ascending: false })
```

- ***** gets every column of the client row.
- **agent:** renames the join so the field is called agent, not user_profiles. **!fkey** names which foreign key to follow, because user_profiles is linked to clients more than once.
- **viewings(count)** asks Postgres to count each client's viewings instead of sending the rows. One request returns everything, so there is no **N+1 problem**.
- Newest first: the newest lead is usually the next call to make.

```
if (error) throw toRequestError(error, "Clients")
```

- Turns Supabase's raw error into the app's error with a readable message.
- Throwing is what makes TanStack Query set error, which DataRenderer turns into the retry message.

```
const rows = (data ?? []) as unknown as (Omit<
  Client,
  "viewing_count"
> & { viewings: { count: number }[] })[]
```

- **Why the double cast:** the Supabase client is created without <Database>, so TypeScript guesses the shape of joins wrongly. TypeScript refuses as between types that do not overlap, so the cast goes through unknown first.
- The type says: a Client without viewing_count (it does not exist yet), plus viewings as an array of { count }, which is how the count really arrives.
- A comment above the cast explains it. The real fix is generated database types (see Still open).

```
return rows.map(({ viewings, ...client }) => ({
  ...client,
  viewing_count: viewings[0]?.count ?? 0,
}))
```

- Destructuring with a **rest property** pulls viewings out and keeps everything else in client.
- The result is a flat viewing_count number. No component has to know that Supabase returns counts as [{ count }].
- **Shaping data at the boundary:** the API converts the database shape into the app's shape once.

createClientApi

```
const { params, user } = await action({
  params: values,
  schema: clientSchema,
  authorize: true,
})
```

- action() validates and transforms the values with Zod: empty strings become null, budgets become numbers, the email becomes lowercase.
- authorize: true loads the signed-in user and throws if there is none.

```
const { error } = await supabase
  .from("clients")
  .insert({
    ...params,
    status: "active",
    assigned_agent_id: user!.id,
  })
```

- The form never sends status or the owner. They are set here, so nobody can create a client as closed or assign it to another agent from the browser.
- user! is safe: authorize: true has already thrown if there is no user.
- RLS on the insert is the real guard. This is **defense in depth**: validation for a good experience, the database for security.

updateClientApi, moveClientApi, deleteClientApi

```
const { data, error } = await supabase
  .from("clients")
  .update(params)
  .eq("id", id)
  .select("id")

if (error) throw toRequestError(error, "Client")

// RLS blocks updates silently: no error, zero rows changed
if (data.length === 0) {
  throw new ForbiddenError("You can only edit your own clients")
}
```

- **The most important pattern in this file.** When RLS blocks an update or a delete, Postgres returns no error: it simply changes 0 rows.
- Without .select("id") and the length check, an agent editing someone else's client would see "Client updated" while nothing changed.
- moveClientApi and deleteClientApi use the same check with their own messages.

```
// With ON DELETE RESTRICT, a client with viewings fails
// here (23503), and toRequestError turns that into
// "This client is linked to other records"
```

- The viewings table points to clients with ON DELETE RESTRICT, so Postgres refuses to delete a client who has viewings. 23503 is the foreign key violation code.
- The UI already hides Delete for those clients. The database is the second layer, and the one that cannot be bypassed.

4. The five data hooks

useClients

```
export function useClients() {
  const { isLoading, data: clients, error, refetch } =
    useQuery({
      queryKey: ["clients"],
      queryFn: getClientsApi,
    })

  return { isLoading, clients, error, refetch }
}
```

- The query key ["clients"] is the cache identity. Every hook that changes clients invalidates this key.
- No role in the key: RLS already decides which rows come back, and the cache is cleared on logout.
- Renaming data to clients makes components read clearly.
- **Shared cache:** ClientsBoard and ViewingForm (the client picker) both call useClients. The second caller gets the cached list instantly.

The mutation pattern (all four hooks)

```
export function useCreateClient() {
  const queryClient = useQueryClient()

  const { isPending, mutate: createClient } = useMutation({
    mutationFn: createClientApi,
    onSuccess: () => {
      toast.success("Client added")
      queryClient.invalidateQueries({ queryKey: ["clients"] })
      queryClient.invalidateQueries({ queryKey: ["dashboard"] })
    },
    onError: (err) => toast.error(err.message),
  })

  return { isPending, createClient }
}
```

- useQueryClient gives access to the cache.
- mutate: createClient renames mutate, so components read createClient(values).
- **invalidateQueries** marks every matching query as stale and refetches the ones on screen. The board updates with no manual state.
- onError can show err.message directly, because the API layer already wrote a readable message.
- isPending disables buttons, so a double click cannot send two requests.

Which hook invalidates what, and why

Hook	Invalidates	Why
useCreateClient	clients, dashboard	New card; dashboard counts clients per stage
useEditClient	clients, viewings, dashboard	Viewings and the dashboard's upcoming list show the client's name
useMoveClient	clients, dashboard	Stage counts change
useDeleteClient	clients, dashboard	Card gone; counts change

The rule: when you write a mutation, find every query that reads that table, including joins. grep "clients(" in src/api finds them.

useEditClient (fixed in this review)

```
onSuccess: () => {
  toast.success("Client updated")
  queryClient.invalidateQueries({ queryKey: ["clients"] })
  // Viewings and the dashboard's upcoming list
  // show the client's name
  queryClient.invalidateQueries({ queryKey: ["viewings"] })
  queryClient.invalidateQueries({ queryKey: ["dashboard"] })
},
```

- **The bug:** the dashboard query selects client:clients(full_name). Before the fix, renaming a client left the old name on the dashboard until its cache went stale.
- Invalidating ["dashboard"] matches every dashboard key, including ["dashboard", role, userId], because invalidation matches by prefix.

useMoveClient (renamed in this review)

```
export function useMoveClient() {
  ...
  const { isPending, mutate: moveClient } = useMutation({
    mutationFn: moveClientApi,
```

- Was useUpdateClient with updateClientStatusApi. Next to useEditClient, "update" could mean anything.
- The old file needed a comment to explain its name. A name that needs a comment is a name to change.
- The UI already called the action onMove and the toast already said "Client moved", so now the code says the same thing.

5. useClientsBoard.ts

New in this review. It holds everything the board does, so the component holds only markup. Same split as useClientForm and ClientForm.

```
function groupByStage(clients: Client[]) {
  return CLIENT_STAGES.map((stage) => ({
    ...stage,
    clients: clients.filter((c) => c.status === stage.status),
  }))
}
```

- Starts from CLIENT_STAGES, so every stage appears in pipeline order, even with zero clients.
- Each stage keeps its status, label and color, and gets its clients.
- Before, the same filter was written twice: once for the phone chips and once for the columns. Now both read the same result.

Why grouping is not in the API

- **The cache is shared.** ViewingForm needs the flat list for its client picker. Grouped data in the API would break it or need a second query for the same rows.
- **The server already filters what matters.** RLS decides which clients you may see. Grouping only decides where each card is drawn: presentation.
- **Server filtering is for less data:** search, pagination, thousands of rows. The board shows every client anyway.
- **Rule:** the API returns the shape every screen needs; anything only one screen needs is derived in that screen's hook.

```
const { user } = useUser()
// Agents only see their own clients,
// so the agent line tells them nothing
const showAgent = user?.user_profile?.role !== "agent"
```

- A derived value: computed from the user on every render, never stored in state.

```
const [editTarget, setEditTarget] = useState<Client>()
const [deleteTarget, setDeleteTarget] = useState<Client>()
const [phoneStage, setPhoneStage] =
  useState<ClientStatus>("active")
```

- **One state, two jobs:** undefined means closed; a client means open, for that client. Separate isOpen and selectedClient states could disagree; one cannot.
- phoneStage is plain UI state, so useState is right. Nothing needs to go in the URL or the cache.

```
function confirmDelete() {
  if (!deleteTarget) return
  deleteClient(deleteTarget.id, {
    onSuccess: () => setDeleteTarget(undefined),
  })
}
```

- The guard clause makes TypeScript sure deleteTarget exists.
- onSuccess passed to the call runs only for this call. The dialog closes only if the delete worked; on failure it stays open and the toast shows why.

```
return {
  clients,
  stages: groupByStage(clients ?? []),
  ...
  openEdit: (client: Client) => setEditTarget(client),
  closeEdit: () => setEditTarget(undefined),
  ...
}
```

- The hook returns intentions (openEdit, closeEdit), not setters. The component says what should happen; the hook decides how.
- If editing ever needs to keep the client while the sheet closes, only the hook changes.

6. ClientsBoard.tsx

```
<div className="min-w-0 space-y-6">
```

- Flex and grid children default to min-width: auto, so one long email would make the page wider than the screen. min-w-0 lets truncate work.

```
<PageHeader
  title="Clients"
  count={clients?.length ?? 0}
  isLoading={isLoading}
/>
```

- ?? falls back only on null or undefined, never on 0. Here || would also work, but ?? is the habit to keep.

```
<CreateButton label="Add client" ...>
  {(onClose) => <ClientForm onClose={onClose} />}
</CreateButton>
```

- children as a function: a **render prop**. CreateButton owns the open state and hands the form a way to close it.

```
<DataRenderer
  data={clients}
  isLoading={isLoading}
  error={error}
  onRetry={() => refetch()}
  loading={<ClientsSkeleton />}
  empty={EMPTY_CLIENTS}
  render={() => ( ...the board... )}
/>
```

- DataRenderer handles loading, error and empty from the flat list. render draws the grouped stages from the hook.
- **onRetry={() => refetch()}:** the arrow matters. onRetry={refetch} would pass the click event in as refetch's options.

The phone stage chips

```
<div className="grid grid-cols-2 gap-2 md:hidden">
  {stages.map((stage) => {
    const selected = stage.status === phoneStage
    return (
      <button
        key={stage.status}
        type="button"
        aria-pressed={selected}
        onClick={() => setPhoneStage(stage.status)}
        className={cn("flex h-10 min-w-0 items-center " +
          "justify-between ...", selected ? "..." : "...")}
      >
        <span className="truncate">{stage.label}</span>
        <span className="tabular-nums">
          {stage.clients.length}
        </span>
      </button>
    )
  })}
</div>
```

- **A 2x2 grid, fixed in this review.** It used to be a scrolling row, and the fourth stage (Inactive) sat completely off-screen with no hint that it existed.
- Both edges now line up with the cards, at every phone width.
- **aria-pressed** marks a toggle button, so a screen reader hears which stage is selected.
- type="button" stops it from submitting a form if one is ever placed around it.
- tabular-nums gives every digit the same width, so counts do not wobble.

The board

```
<div className="grid grid-cols-1 items-start gap-4
  md:grid-cols-2 xl:grid-cols-4">
  {stages.map((stage) => (
    <section
      key={stage.status}
      aria-label={stage.label}
      className={cn(
        "min-w-0 rounded-xl bg-muted/50 p-3",
        stage.status !== phoneStage && "hidden md:block"
      )}
    >
```

- One column on phones, two on tablets, four on wide screens.
- **items-start:** without it, every column stretches to the tallest one and leaves empty grey boxes.
- **hidden md:block:** phones show only the selected stage; tablets and up show all of them. Done in CSS, so there is no useMediaQuery hook and no flash of the wrong content.
- Each column is a labelled section, so screen reader users can jump between stages.

```
<span aria-hidden="true" className="size-2 rounded-full"
  style={{ backgroundColor: stage.color }} />
```

- The color dot is decoration, so aria-hidden. The label carries the meaning.
- Inline style because the color comes from data; Tailwind cannot build classes at runtime.

```
{stage.clients.length === 0 ? (
  <EmptyMessage empty={EMPTY_STAGE} compact />
) : (
  <ul className="space-y-3">
    {stage.clients.map((client) => (
      <li key={client.id}>
        <ClientCard
          client={client}
          showAgent={showAgent}
          onEdit={() => openEdit(client)}
          onMove={(status) =>
            moveClient({ id: client.id, status })}
          onDelete={() => openDelete(client)}
        />
      </li>
    ))}
  </ul>
)}
```

- An empty stage says so instead of showing nothing.
- key={client.id}: the database id is unique and stable, unlike the index.
- **The card does not know about mutations.** It receives callbacks. The board decides what moving or deleting means. The card is easy to reuse and test.

The edit sheet and the delete dialog

```
<FormSheet
  open={Boolean(editTarget)}
  onOpenChange={(open) => !open && closeEdit()}
  title="Edit client"
>
  {/* Mounts a fresh form per client:
      useForm reads defaultValues once */}
  {editTarget && (
    <ClientForm client={editTarget} onClose={closeEdit} />
  )}
</FormSheet>
```

- **A controlled component:** the sheet's open state comes from editTarget.
- onOpenChange fires when the sheet wants to close (Esc, overlay, X). We only react to closing; opening happens through openEdit.
- **The key line:** useForm reads defaultValues only when it mounts. Unmounting the form between edits gives each client a fresh form. Without it, editing A then B would show A's data.

```
<ConfirmDelete
  open={Boolean(deleteTarget)}
  onOpenChange={(open) => !open && closeDelete()}
  onConfirm={confirmDelete}
  isPending={isDeleting}
  title="Delete this client?"
  description="Only for clients added by mistake.
    This can't be undone."
/>
```

- The description tells the user what delete is for. Closing a lead is done by moving it to a stage.
- isPending shows progress and blocks a second click.

7. ClientsSkeleton.tsx

```
<div role="status" className="space-y-6">
  <span className="sr-only">Loading clients</span>

  {/* Phone: the 2x2 stage chips */}
  <div className="grid grid-cols-2 gap-2 md:hidden">
    {SKELETON_KEYS.slice(0, 4).map((key) => (
      <Skeleton key={key} className="h-10 rounded-full" />
    ))}
  </div>

  <div className="grid grid-cols-1 gap-4
    md:grid-cols-2 xl:grid-cols-4">
    {SKELETON_KEYS.slice(0, 4).map((key, column) => (
      <div key={key} className={column === 0
        ? "space-y-3" : "hidden space-y-3 md:block"}>
        ...
```

- **Same shape as the real board**, so nothing jumps when data arrives. That jump is a **layout shift**.
- The chip row was missing before this review, so on phones the board jumped down when the data loaded.
- SKELETON_KEYS gives stable keys without using the index.
- role="status" with sr-only text: screen readers hear "Loading clients".

8. ClientCard.tsx

```
// "Penthouse · Downtown Dubai, DIFC"
function describeWants(client: Client) {
  const type = propertyTypes.find(
    (t) => t.value === client.preferred_type
  )?.label
  const areas = (client.preferred_locations ?? []).join(", ")
  return [type, areas].filter(Boolean).join(" · ")
    || "No preferences yet"
}
```

- The database stores penthouse; the card shows Penthouse. Labels live in one constants list, shared with the form's select.
- **filter(Boolean)** drops empty parts, so there is never a dangling " · ".
- Here || is right: an empty string should fall back too.
- A pure helper outside the component: it does not need state, and it is not recreated in a way that matters.

```
const viewingsLabel =
  viewings === 0
    ? "No viewings"
    : `${viewings} viewing${viewings === 1 ? "" : "s"}`
```

- "No viewings", "1 viewing", "3 viewings". Small, but reviewers notice singular and plural.

```
<article className="rounded-lg border bg-card p-4">
  <div className="flex items-start gap-2">
    <div className="min-w-0 flex-1">
      <h3 className="truncate font-medium">
        {client.full_name}
      </h3>
```

- article: a self-contained item inside the board's list.
- min-w-0 flex-1 lets the name truncate instead of pushing the menu button off the card.

```
<dl className="mt-3 space-y-1 text-sm">
  <dt className="sr-only">Looking for</dt>
  <dd className="truncate">{describeWants(client)}</dd>
  <dt className="sr-only">Budget</dt>
  <dd className="... tabular-nums">
    {formatBudgetRange(client.budget_min, client.budget_max)}
  </dd>
</dl>
```

- A **description list**: label and value pairs. Screen readers hear "Looking for: Penthouse".
- The labels are visually hidden because sighted users understand from the layout.

```
<a href={`mailto:${client.email}`}
  className="flex min-h-9 items-center gap-2 ...">
  <Mail className="size-4 shrink-0" />
  <span className="truncate">{client.email}</span>
</a>
{client.phone && (
  <a href={`tel:${client.phone.replace(/\s/g, "")}`} ...>
```

- One tap starts an email or a call on a phone. The card is a tool, not just a display.
- replace(/\s/g, "") removes spaces, because some dialers break on them.
- min-h-9 (36 px) makes the links easy to hit with a thumb.
- shrink-0 on the icon stops it from being squeezed by a long email.

```
<div className="mt-2 flex items-center gap-2 border-t pt-3">
  {showAgent && (
    <span className="flex min-w-0 items-center gap-2">
      <UserAvatar name={client.agent?.full_name}
        src={client.agent?.avatar} size="xs" />
      <span className="truncate">
        {client.agent?.full_name ?? "No agent"}
      </span>
    </span>
  )}
  <span className="ml-auto shrink-0 tabular-nums">
    {viewingsLabel}
  </span>
</div>
```

- **Changed in this review:** an empty used to sit in place of the agent line, only so justify-between would push the count right. ml-auto does that with no empty element.
- agent?. and ?? "No agent": the join can return null if the profile is gone.

9. ClientActions.tsx

```
interface ClientActionsProps {
  client: Client
  onEdit: () => void
  onMove: (status: ClientStatus) => void
  onDelete: () => void
}
```

- **Changed in this review:** the file used to declare a copy called ClientCardProps, including showAgent, then removed it with Omit. A component declares the props it uses, under its own name.

```
// Clients with viewings are records:
// close them with a stage, not a delete
const canDelete = client.viewing_count === 0
const otherStages = CLIENT_STAGES.filter(
  (s) => s.status !== client.status
)
```

- canDelete is the user-experience layer; ON DELETE RESTRICT is the security layer.
- The Move to list never offers the stage the client is already in.

```
<DropdownMenuTrigger asChild>
  <Button size="icon" variant="ghost"
    aria-label={`Actions for ${client.full_name}`}>
    <MoreHorizontal />
  </Button>
</DropdownMenuTrigger>
```

- asChild: Radix passes its behaviour to our Button instead of wrapping it in another button.
- An icon-only button needs a name. With the client's name, a screen reader does not hear ten identical "Actions" buttons.

```
<DropdownMenuItem onSelect={() => onMove(stage.status)}>
```

- onSelect, not onClick: Radix fires it for mouse, Enter and Space, then closes the menu.

10. useClientForm.ts

```
const EMPTY_VALUES: ClientFormValues = {
  full_name: "",
  email: "",
  phone: "",
  budget_min: "",
  ...
  preferred_locations: [],
}
```

- Inputs work with strings, so empty means "", not null or undefined. That keeps every input **controlled** from the first render.

```
// Database values (numbers, nulls) to form values (strings)
function toFormValues(client: Client): ClientFormValues {
  return {
    full_name: client.full_name,
    phone: client.phone ?? "",
    budget_min: client.budget_min?.toString() ?? "",
    ...
  }
}
```

- **Two directions, each handled once.** toFormValues turns database values into form values. clientSchema.transform turns them back on save ("" becomes null, "500000" becomes 500000).
- budget_min?.toString(): optional chaining, because the budget can be null.

```
const form = useForm<ClientFormValues>({
  resolver: zodResolver(clientFormSchema),
  mode: "onTouched",
  defaultValues: client ? toFormValues(client) : EMPTY_VALUES,
})
```

- zodResolver connects the Zod schema to React Hook Form, so the same rules run in the form and in the API.
- **mode: "onTouched":** no red errors while typing the first time. A field is checked when you leave it, then live after that.
- One form for add and edit, chosen by whether a client was passed in.

```
const onSubmit = form.handleSubmit((values) => {
  if (client) {
    editClient({ id: client.id, values }, { onSuccess: onClose })
  } else {
    createClient(values, { onSuccess: onClose })
  }
})

return {
  form,
  isEdit: Boolean(client),
  isPending: isCreating || isEditing,
  onSubmit,
}
```

- handleSubmit runs validation first and only calls our function with valid values.
- The sheet closes only on success. On error it stays open with the user's input still there.
- The footer only needs to know "busy or not", so the two pending flags become one.

11. ClientForm.tsx

```
interface ClientFormProps {
  // No client: add mode. A client: edit mode
  client?: Client
  onClose: () => void
}
```

- **Moved in this review** from types/database.d.ts. That file describes database rows; component props live next to their component.

```
<form onSubmit={onSubmit} noValidate
  className="flex flex-1 flex-col overflow-hidden">
  <div className="flex-1 space-y-6 overflow-y-auto px-6 py-6">
    ...fields...
  </div>
  <FormFooter ... />
</form>
```

- **noValidate** turns off the browser's own popups, so only Zod's messages show, and they look the same in every browser.
- The body scrolls (flex-1 overflow-y-auto) and the footer sits outside it, so Save stays pinned at the bottom of the sheet.

```
<section className="space-y-5">
  <h3 className="text-sm font-medium">Contact</h3>
  <TextField control={control} name="full_name"
    label="Full name" icon={UserIcon} required />
  <TextField control={control} name="email"
    type="email" ... required />
```

- Two sections, Contact and What they are looking for. Grouping makes a long form feel short.
- type="email", type="tel" and type="number" give phones the right keyboard.
- required and optional only change the label. The rules live in the schema.

```
<FormFooter
  canSave={!isPending}
  isSubmitting={isPending}
  onCancel={onClose}
  submitLabel={isEdit ? "Save changes" : "Add client"}
  submittingLabel={isEdit ? "Saving…" : "Adding…"}
/>
```

- The button names what will happen. "Save changes" and "Add client" are clearer than "Submit".
- The pending label tells the user the click was received.

12. Decisions and changes

Use these as STAR stories: situation, task, action, result.

A stale name on the dashboard

When I reviewed my mutations, I listed every query that reads the clients table. The dashboard's upcoming viewings join client names, but editing a client never invalidated the dashboard, so a renamed client showed the old name. I added the invalidation, and now I check joins whenever I write a mutation.

A name that needed a comment

My hook for moving a client between stages was called useUpdateClient, next to useEditClient, and it needed a comment to explain itself. I renamed it useMoveClient, and the API function moveClientApi, so the code uses the same word as the UI and the toast.

Logic out of the component

The board component mixed data, mutations, sheet state and markup. I extracted useClientsBoard, the same split I use for forms. The component is now only layout, and the hook returns intentions like openEdit instead of setters.

Where filtering belongs

I usually keep data work in the API layer, so I considered grouping by stage there. But the viewing form shares the same cached list and needs it flat, and the server already does the filtering that matters through RLS. Grouping is presentation, so it lives in the board's hook.

A hidden stage on phones

On a phone, the stage chips scrolled sideways and the fourth stage sat fully off-screen with no hint. I switched to a 2x2 grid, so all four stages are always visible, and matched the skeleton so the page does not shift on load.

Silent RLS blocks

Postgres returns no error when RLS blocks an update. It just changes zero rows. Every update and delete selects the id and throws a ForbiddenError when nothing changed, so the user never sees a false success.

13. Still open

Item	Why it matters
as unknown as cast	The Supabase client is untyped. createClient<Database> with generated types would type the joins and remove the cast
Edit sheet empties while closing	closeEdit sets editTarget to undefined at once, so the form disappears during the slide-out animation. Keep the last client while closing
Other prop types in database.d.ts	ViewingFormProps and others move next to their components in their own branches
No search or filter	Fine for a demo agency. With hundreds of clients, search should run on the server with pagination
Demo data	"Jane Deo" and similar typos look careless to recruiters

14. Interview questions

Q1. Walk me through how the clients page loads.

The page renders ClientsBoard, which calls useClientsBoard. That hook calls useClients, a TanStack Query with the key ["clients"]. Its function, getClientsApi, selects clients with the agent joined and a viewings count, then flattens the count. The hook groups clients by stage, and the component only draws them.

Q2. How do you get each client's viewing count without an N+1 problem?

I select viewings(count) in the same query, so Postgres counts per client and returns the numbers with the rows. One request instead of one per client. The API then flattens it into viewing_count.

Q3. RLS blocks an update. What happens, and how do you handle it?

Postgres does not return an error. The update simply affects zero rows. I add .select("id") and throw a ForbiddenError when the result is empty, so the user sees a clear message instead of a false success.

Q4. What does invalidateQueries do?

It marks matching queries as stale and refetches the ones currently on screen. After a mutation I invalidate every query that reads the changed data, and the UI updates without manual state.

Q5. You found a bug where the dashboard showed an old client name. Explain it.

The dashboard's upcoming viewings join the client's name, but editing a client only invalidated clients and viewings. The dashboard kept its cached name until it went stale. I added the dashboard invalidation. The lesson: check joins, not just the table you changed.

Q6. Why is grouping by stage in a hook and not in the API?

The flat list is shared through the cache: the viewing form needs it for its client picker. RLS already does the security filtering on the server. Grouping is presentation that only the board needs, so it belongs in the board's hook.

Q7. When would you filter on the server instead?

When it reduces the data sent: search, pagination, or large tables. Showing every client on one board means fetching them all anyway.

Q8. Why extract useClientsBoard?

To separate logic from markup. The hook owns data, mutations and state; the component only lays things out. It is easier to read, and the logic can change without touching the JSX.

Q9. Why does the hook return openEdit instead of setEditTarget?

The component should say what happens, not how. If the implementation changes, for example keeping the client during the close animation, only the hook changes.

Q10. Why is the edit form only mounted when there is an edit target?

useForm reads defaultValues once, when it mounts. Mounting a fresh form per client guarantees the right values. Otherwise editing a second client would show the first client's data.

Q11. How do you stop users deleting clients that have viewings?

Two layers. The UI hides Delete when viewing_count is above zero. The database has ON DELETE RESTRICT, so even a direct API call fails with 23503, which I turn into a readable message.

Q12. Why one form for add and edit?

Both change the same fields with the same rules. One component with an optional client avoids two forms drifting apart. The labels and the submit action change based on isEdit.

Q13. Why convert values in toFormValues and in the schema?

Inputs work with strings, the database with numbers and null. toFormValues converts database values to form values once, and the Zod transform converts them back once. Each direction lives in one place.

Q14. Why mode: "onTouched"?

No errors while the user is typing a field for the first time. It validates when they leave the field, then live after that. Less shouting, still fast feedback.

Q15. How does the board work on phones?

A 2x2 grid of stage chips picks one stage, and CSS hides the other columns below the md breakpoint. On tablets and up, all columns show. No JavaScript media query, so there is no flash.

Q16. What is aria-pressed, and why use it?

It marks a button as a toggle and says whether it is on. My stage chips use it, so a screen reader announces which stage is selected.

Q17. Why does the actions button have an aria-label with the client's name?

Icon-only buttons have no text, so they need a name. Including the client's name means a screen reader user can tell ten action buttons apart.

Q18. Why onRetry={() => refetch()} and not onRetry={refetch}?

refetch accepts options. Passing it directly would send the click event in as those options. The arrow function calls it with no arguments.

Q19. What would you improve next?

Type the Supabase client with generated database types to remove the cast, and keep the client in the edit sheet during its close animation.

15. Vocabulary

Term	Meaning
Pipeline	The stages a client moves through, from first contact to closed
N+1 problem	One query for a list, then one more per item
Join	Fetching related rows through a foreign key in one request
Rest property	...rest in destructuring: everything not named
Shaping at the boundary	Converting the database shape to the app's shape in the API layer
Invalidation	Marking cached queries stale so they refetch
Shared cache	Two components using the same query key get the same data
Silent RLS block	An update or delete that affects zero rows with no error
ON DELETE RESTRICT	A foreign key rule that refuses to delete a referenced row
Defense in depth	Several layers of protection: UI, validation, database
Derived value	Computed during render instead of stored in state
Controlled component	Its value or open state comes from props or state
Render prop	A function prop or child that returns UI
Layout shift	Content jumping when data arrives
Toggle button	A button with an on and off state (aria-pressed)
Description list	dl, dt, dd: label and value pairs

16. Self-test

Say each answer out loud in under 45 seconds, without looking.

1. What does the clients page do, and what are the four stages?
2. Walk me through how the clients are loaded.
3. What does viewings(count) do, and why?
4. Why the double cast in getClientsApi?
5. Why does the insert set status and the owner itself?
6. How do you catch a silent RLS block?
7. What does ON DELETE RESTRICT do here?
8. What does invalidateQueries do?
9. Explain the dashboard stale-name bug and its fix.
10. Why rename useUpdateClient to useMoveClient?
11. Why is groupByStage in the hook, not the API?
12. Why extract useClientsBoard?
13. Why does the edit form mount only when there is a target?
14. Why did the phone chips become a 2x2 grid?
15. What do toFormValues and the schema transform do?
16. Why mode: "onTouched"?
17. Name two accessibility details on the card.
18. What would you improve next?